Reproducible Science: Machine-Checkable Claims from Preregistration to Publication

Elliot Tower
elliot@elliottower.ai

Abstract

Every claim in a paper comes from somewhere. AI tooling allows us to quickly generate hypotheses, run analyses, and draft manuscripts, but we identify verification and provenance as open issues: (1) verifying that the finished paper agrees with the artifacts behind it typically requires manual or agentic verification, which is expensive and difficult to audit; (2) verification usually does not include a denominator for the number of sources or artifacts checked; (3) verification snapshots quickly become stale, and are not guaranteed to find every issue or reproduce the same issues on subsequent runs; (4) more experiments means more researcher degrees of freedom and post-hoc analysis, even if unintentional, or done by an agent; and (5) verification is typically done post-hoc, and does not cover all steps of the research lifecycle.

We present Reproducible Science[1], a declarative, continuous, machine-checkable provenance layer for research claims, throughout the whole research lifecycle. `prereg` freezes a plan before the run, `results` seals its inputs, records its outputs and binds numbers in the manuscript to the run that produced them, `citations` pins exact quotations inside cited sources, and `repro` checks every declaration against the artifact it names. Our Claude Code plugin records provenance at the time of experimentation and reports drift on every edit through hooks. Applying this contract to this paper[2] surfaced two errors, one of which had already gone stale twice. As agents take over more of the research lifecycle, the artifacts they leave behind are worth only as much as we can verify the claims inside them.

1 Introduction

Surveys of research agents name verification as the step that has not kept pace [Ding et al., 2026]. Re-running the analysis is the usual answer, and it is usually unavailable: across 863,878 attempted executions of valid notebooks, 24.11% ran without errors and 4.03% produced the results the notebook recorded [Pimentel et al., 2019]. The deeper problem is that re-running checks the wrong object. A notebook that re-executes perfectly can still sit beside a manuscript printing a value no output contains, because execution establishes that the code runs and says nothing about what the paper claims the code produced. Reproducibility and verification are different properties. A project can preserve its code, its data and its logs and still print a value no output contains, quote a passage that was never read out of its source, or call an analysis confirmatory after its plan changed.

We call what accumulates **verification debt**. Like debt in software, it is cheapest to detect when it is introduced and expensive to reconstruct afterwards. At the moment a value changes, the run that produced the old one is still on disk, the source still resolves, and the edit carries an author and a commit. Later the run has been overwritten, the source was fetched once without its address recorded, and which document a citation referred to is no longer recoverable from the citation. In 14

[1]Package: https://github.com/elliottower/reproducible-science
[2]Paper code and artifacts: https://github.com/elliottower/reproducible-science-paper

Accepted at the AI for Science workshop (NeurIPS 2026).

of 34 documented cases of retraction, correction, or misconduct, the checks of Reproducible Science would flag a stated problem, including both Surgisphere COVID-19 studies (Appendix A).

Table 1: Three boundaries of a research project, the failure each admits when nothing checks it, and the comparison Reproducible Science performs. All three are decided the same way, by a digest of a file and a comparison at a location the author declares in it, so one checking engine serves them.

Boundary	**What goes wrong when it is unchecked**	**What is compared**
plan to run	a hypothesis or an analysis is rewritten after the outcome is visible	a run record against a plan pinned by digest
run to manuscript	a printed value drifts from the output it was read from	a declared address in a sealed result file
source to manuscript	a quotation or an identifier does not occur in the artifact cited for it	a passage against a content-addressed source

Most of what a reader would want checked is not mechanically checkable: whether a design answers the question, whether a conclusion follows. A narrow part is. A number printed in a manuscript either matches the results file it came from or does not, and a quoted sentence either occurs in the cited source or does not. That part can be settled without a human and without a model, at a cost small enough to run on every commit rather than once before submission.

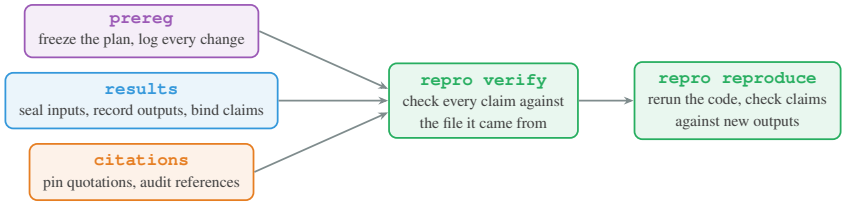


Figure 1: The commands of Reproducible Science. `prereg`, `results` and `citations` write declarations as the work is done; `repro verify` checks each against its artifact, and `repro reproduce` against rerun outputs.

Nothing failed because nothing ran. In our own code, a missing extractor, the program that pulls text out of a PDF, returned the empty string, the check recorded an unreadable document, and the run summary read *nothing failed*. A check that read no text and a check that read the text and found it correct produced the same line. A separate incident is worse: a script collected the digests cited by every registration under one directory and verified each against a file; the directory was renamed, the glob matched nothing, the script exited zero, and continuous integration, a pre-push hook and a make target were all green in that state. A report that returns success is not evidence that anything was examined, and an outcome vocabulary with no term for non-execution cannot say so.

Position. Three claims follow, each against a live alternative. *Prospective decisions require prospective records*: a frozen plan cannot establish that an author was honest, but it makes subsequent rewriting detectable and separates confirmatory work from exploratory work. *Evidence requires declared addresses*: a reported `0.42` should name the pointer or selector it came from, because searching afterwards for a numerically similar value is not provenance and does not scale. The alternative is to recover the address afterwards, and a contemporaneous system does: Paperclip binds the same unit, a claim to a location in a source, and verifies it with a language-model subagent against line ranges into its own re-parse [Generative Expert Labs, 2026]. A verdict from a model is not recomputable from the artifact, and an address into a re-parse moves when the parser improves. We make the opposite choice on both: the comparison is deterministic, and an address points into pinned bytes. *Verification requires an explicit non-execution state*: a checker that cannot distinguish matched, disagreed, absent and could-not-run has no way to report that a green build examined nothing. Our own infrastructure violated the non-execution requirement twice.

Contributions. (1) **Lifecycle verification**: a model spanning the plan-to-run, run-to-manuscript and source-to-manuscript boundaries of Table 1, designed to run while a project changes. (2) **Evidence assertions**: typed assertions binding manuscript values and quotations to declared locations in content-addressed artifacts, with four implemented variants over a shared engine and report.

(3) **Staged outcomes**: a deterministic model separating execution, extraction and comparison, with a project's pass-or-fail policy applied to the resulting facts afterwards. We instantiate all three in Reproducible Science (Figure 1) and evaluate them through conformance tests, fault injection, a quotation audit, defects met in ordinary use, and an audit of a project developed with it.

2 Model and Contract

Section 1's two incidents fail differently — one check never ran, the other ran and read no text — and both reported what a passing check reports. The three boundaries of Table 1 share one architecture: declare an address, pin the artifact it points into, compare deterministically, and report each stage separately. Two of the three are decided by an *evidence assertion*, which binds a manuscript claim to an artifact, a locator within it, and the value or passage expected there. A project lists its assertions in one file, the manifest. The plan-to-run boundary is decided by reading a run record beside a pinned plan, so no evidence assertion is defined for it. Four variants are implemented across the other two boundaries. They differ in how evidence is located and compared, and share artifact pinning, staged outcomes and one report. Appendix B provides the full definition of each term used here.

Table 2: The four implemented variants. A table cell is addressed by a selector over key columns, because a row index addresses a different cell when the table is reordered. A correspondence names no expected value: neither side is the reference, and a disagreement prints both readings.

Kind	Assertion	Located by	Compared how
metric	this artifact holds this value here	RFC 6901 JSON Pointer	decimal, at the precision the manuscript printed
table	this table holds this value in this cell	column plus a row selector	decimal, at the precision the manuscript printed
quote	this passage occurs in this artifact	page, optionally	verbatim after normalization that preserves which words appear
correspondence	two artifacts hold the same value	a locator on each side, prose anchors or a pointer	decimal, at the precision the manuscript printed

Three stages. A check must execute, extract, and compare, and the three fail independently. Table 3 gives the outcomes and the flattened view derived from them. The rows differ in what a reader should do: revise the claim, fix the pointer or run the analysis that produces the value, install a package, file a bug. A report that cannot distinguish them directs none of those actions.

Table 3: The three stages and the flattened view derived from them, the single label a report prints. A pointer to a key the results file lacks is reported `not_found`: nothing is asserted about whether the value differs. An extractor that is not installed yields `unchecked`, and an extractor that raises yields `error` and never `unchecked`.

Situation	Execution	Extraction	Comparison	Flattened
value present	completed	extracted	match	`verified`
value differs in a readable artifact	completed	extracted	mismatch	`mismatch`
pointer does not resolve	completed	absent	—	`not_found`
extractor not installed	unavailable	not attempted	—	`unchecked`
backend defect	failed	not attempted	—	`error`

Pins. Artifacts are content-addressed, and pins are resolved before any evidence is read, so a source that moved is known before decisions are computed against it. Decisions against a broken or unpinned artifact still run and are reported, marked non-authoritative, because a diagnostic is more useful than a blank.

Facts, then policy. Verification returns facts. Whether they constitute a pass depends on what the project is for: an unchecked citation is acceptable in a draft and disqualifying in a submission, and both readings are correct about the same run. The engine therefore reports the three stages and computes no verdict, and a profile maps outcomes to severities (Table 13, Appendix C).

3 Implementation and Workflow

Reproducible Science runs on ordinary files. It does not require an experiment to be conducted in a particular language, notebook system, analysis framework or agent runtime, and it does not require a manuscript to be authored inside its toolchain. The fifteen papers in the quotation audit of Section 4.2 were written without this contract in mind, and none was modified to be auditable. A value is read from nine formats, such as JSON and Parquet, and a quotation from five, such as PDF.

A new or existing project adopts Reproducible Science with one command, `repro init`. A project declares as it goes. A plan is frozen before the run it governs, against a commit and a content hash. The script is committed, its inputs are sealed before an analysis and its outputs recorded after it, so every result can be regenerated: `repro reproduce` runs the declared command again and checks the manuscript's assertions against the file it writes. Misconduct that alters a reported result now requires more than an edit to the manuscript: the committed script must produce the altered value from the sealed inputs, and a re-run by a reader reveals any inconsistency. For each sentence that prints a number, the manifest records the address the number came from, written when the sentence is. Verification reads the declarations and reports on each.

Table 4: The commands and what each establishes. The first three write declarations and `repro` reads them, applying a policy to the result.

Command	What it establishes
`prereg`	that the plan is unchanged since it was frozen, against a commit and a content hash
`results`	that a reported number came from a recorded run over sealed inputs
`citations`	that a quoted passage occurs in the source cited for it
`repro verify`	that every claim matches the file it came from, and which do not
`repro reproduce`	that every claim matches the new output when its command is rerun

A worked example. Appendix C states that the conformance suite holds 25 fixtures, and this paper's manifest binds that sentence with two assertions: a `quote` assertion checking that the manuscript still prints it, and the `metric` assertion below, checking that the pinned inventory of the suite holds the number at the declared pointer.

```
- kind: metric
  artifact: conformance     # pinned: sha256 b9840b9d...
  pointer: /fixtures
  reported: '25'

$ repro verify --policy strict
  ok    conformance-fixtures metric   /fixtures = 25
  72 verified
  policy strict: passed  (0 errors, 0 warnings)
```

With `reported: '19'`, the stale count of Section 4.1, the line is marked `MISS`, the run exits 1, and the report reads *manuscript prints 19, conformance_inventory.json holds 25*. The Claude Code plugin prompts for such declarations while the artifacts are produced: after each edit it names any number or quotation in a manuscript that nothing yet backs, and before an analysis runs it says when the governing plan is unfrozen. A companion Claude Code mod shows the state of a project's records in the session, one row per command, with a warning when a frozen plan was edited or a run was recorded with nothing sealed (Appendix D, Figure 2).

Preregistration is optional and per claim. A claim is marked confirmatory, exploratory, or not applicable, and only the first requires a plan behind it. This matters for the honesty of the report: a project that registers nothing is not failed for it, and a project that calls a claim confirmatory is held to the plan it named.

Cost is what makes the check a commit-time one. Verifying this paper's 76 assertions over 25 artifacts takes 0.53 seconds; resolving its 63 quotations against 17 content-addressed sources takes 4.10 seconds (Appendix E).

4 Evaluation

The conformance suite and the fault injections are in Appendix C: 25 fixtures, each recording its expected outcome, reason and artifact validity as a triple so a case cannot pass by reaching the right verdict for the wrong cause, and four injected faults each distinguishing a class the flattened view would otherwise merge.

4.1 Self-audit

This paper is an instance of what it describes. Each number its manifest binds carries two assertions: that the manuscript states the sentence, and that a pinned artifact holds the value at a declared pointer. Thirty-eight claims are bound this way, 76 assertions over twenty-five artifacts, and they pass under the `strict` policy profile. The manifest and the artifacts it pins are in the repository, so the check is one a reader runs rather than a result they take on trust. No claim here is confirmatory, so none requires a registered plan.

Building the manifest surfaced two defects in this paper's own numbers. Four counts in Table 12 had been computed correctly and written to no file, so nothing could re-check them; derived into an artifact, all four match what the manuscript prints. The count of conformance fixtures was recorded and stale: a stored figure gave nineteen where the suite holds 25, and the specification had earlier stated eighteen against a count of nineteen.

4.2 Quotation audit

Six extractors read the same 2,364 passages, quoted from fifteen published papers, out of the same pinned bytes and returned six different answers, from 2,314 down to 1,716 (Table 12). A verification snapshot therefore records the reader as much as the corpus, and a rerun under a different toolchain is not guaranteed to find the same issues. Under poppler's layout mode 113 quotations fail to resolve and every failure traces to extraction (Appendix C), so a checker with only pass and fail would have accused the manuscripts 113 times with no misquotation present.

4.3 Recall against CORE-Bench

The corpora above carry no ground truth: nobody has labelled which of a paper's numbers its artifact really holds, so a located value is evidence of an occurrence. CORE-Bench supplies the missing half, and a bound: a value no reader finds in an artifact is a value no assertion can point to. Each capsule, CORE-Bench's package of a paper's code and data, names the values its analysis produces, so recall is measured against a stated answer set rather than estimated [Siegel et al., 2024].

Over 45 capsules the benchmark states 240 values, of which 146 are located in the capsule's files without running it: a recall of 61%. Table 5 gives it by the strength of the value, where strength is the number of constraining digits. Recall falls as values carry more digits and then rises again, because the tiers differ in what they are made of rather than in difficulty: a two-digit integer is found in almost any artifact and a long decimal is found only where it was written. A second run on a separate machine produces a byte-identical record for all 45 capsules.

Table 5: Recall over 45 CORE-Bench capsules, against the values each capsule states it produces. Eight capsules hold a record no reader here opened, so their misses are `unchecked` rather than `absent`: a two-state report would count all of them against the capsule. This is the same separation Section 4.2's quotation audit makes, on a third party's corpus and against a known answer set.

Strength	Stated	Located	Recall
trivial	59	54	92%
weak	76	40	53%
moderate	44	15	34%
strong	61	37	61%
all	240	146	61%

5 Findings from Deployment

5.1 Defect classes surfaced in ordinary use

Running the four commands over our own corpus during ordinary research surfaced six defect classes. Table 6 lists them. Each is reported as a class that occurs and is mechanically detectable; no frequency is claimed, and none of these counts enters any rate in Section 4. Two of the six are defects in a check rather than in a manuscript, which is the case the three-stage model exists for.

Table 6: Six defect classes surfaced by running the four commands over one author's corpus. Status records whether a change has been made that prevents the class, not whether the instance was repaired.

Class	What is observably wrong	Status
stored value disagrees with the printed value	nine of ten table rows disagree with the file the row is read from	open
false mismatch under float equality	three of thirty-nine agreements hold only under decimal comparison	closed
recorded hash compared against nothing	digests recorded and never compared to the bytes on disk	closed
check that runs on no files	a glob matches nothing, the check exits zero, and the build is green	open
claims separated from the evidence they name	178 assertions marked verified, and no source in any commit	closed locally
reported figure that stopped reproducing	a reported count no longer reproduces from the bibliography it described	open

Two that integrity checking cannot reach. In the first row every artifact resolves at its recorded digest, so no integrity check fires: the files hold the values the run wrote, and the manuscript's table holds different values. A value printed at `0.001` is stored as exactly zero, and one printed at `0.0003` as `9.73e-7`. The disagreement is reachable only by a check that compares a file to the sentence citing it. In the fifth, a repository split carried 178 assertions and left the sources they named behind; the stored `verified` marks survived the split, so the repository reported a fully verified claim set that a reader could recheck in no part. Quotation verification had passed over every one of those sources, because a quotation resolves against the text as it stands.

5.2 What a reader can re-check from a clone

One command, `repro audit`, runs all four commands over a clone of the public repository for a project developed with it, at a pinned commit.[3] The project keeps registered plans, a results ledger and pinned quotations, and declares no evidence manifest. The clone carries 1,267 tracked files, three registered plans, a ledger of 153 events binding 131 manuscript claims to 13 runs, and seven claim records holding 18 quotations. Table 7 gives what each command established from the clone alone and what it could not.

The quotations do not verify from a clone: a publisher's PDF is not the author's to redistribute, so the repository carries only each source's digest, and all 18 quotations come back `unchecked`.

In a preregistered study of drug-trial outcomes, four rounds of journal review raised 45 comments and Reproducible Science reported 31 findings, with no overlap (Appendix I). Reviewers assess the argument; Reproducible Science checks each value against its source.

A certificate of reproduction does not make a paper's numbers checkable. Of 133 certified reproductions in the CODECHECK register [Nüst and Eglen, 2021], 6 leave a file holding one of the paper's tables, and under half the numbers in those tables have a cell to check against (338 of 713). Of the 338, 326 match the certified file and 12 differ (Appendix F).

Reproducible Science also runs under a literature census conducted by a language model and coding agents. The model screens 43,462 abstracts and extracts 14,053 claim records, each carrying the

[3] https://github.com/elliottower/knockout-epistasis-dynamics, commit `cb82384`.

Table 7: The four commands over a clone of the project's public repository. Two establish their result from the clone alone, because the plans and the ledger are committed with it. The quotation check reads no source and reports that, where a two-state report would have to print a pass or a failure.

Command	What it found	What it could not establish
`prereg`	three plans, each unchanged against the digest recorded when it was frozen	whether a plan was written before the outcomes it governs were seen
`results`	the chain is intact and anchored at 153 events; 8 of 12 recorded files match their digest, and 4 carry more than one recorded digest	what share of the manuscript's numbers the 131 claims cover: the manuscript is not in the repository
`citations`	quotations: 18 against 7 sources, none checked; bibliography: 28 of 32 entries checked, 6 differ from their registry record	whether any quotation occurs in its source
`repro`	no evidence manifest; exit 2, nothing addressed	any assertion, none having been declared

sentence it was drawn from. `citations verify` checks every sentence against the stored abstract: 13,989 match exactly, 27 match in two places, and 37 leave out one word (Appendix L).

5.3 An edit to a registered plan

A preregistration carries a recorded digest, and an edit to it is detected by recomputing that digest rather than by asking the author what changed. In ordinary use `prereg check` caught a subagent that had edited a frozen plan partway through an analysis, and the edit was reverted. The Claude Code plugin and mod refuse edits to a frozen plan, so the plan keeps matching the cryptographic hash recorded at its freeze. Changes go in a separate amendment, sealed the same way.

6 Related Work

Claim-to-location binding. Paperclip binds individual claims to individual locations in sources: an agent adds a claim naming a document identifier and a line range, and a commit verifies each claim against the source text, dropping unsupported claims, with a log for audit [Generative Expert Labs, 2026]. The unit is the same as ours and three properties differ. Verification is performed by a language-model subagent, so the verdict is not reproducible from the artifact. Line ranges address the system's own parse of a document, which is refetched and re-read, so an address points into a mutable representation and improving the parse moves every stored range. No digest is recorded, which under Section 2 leaves every such artifact unpinned and every decision against it non-authoritative. A commit also lands when verification is skipped, which is the condition Section 1 describes.

Repository auditing. adduce audits a research repository against 78 rules and returns a score, asking whether "for every number in the paper, can I point to the artifact that produced it, and will that artifact still produce it elsewhere?" [Chudasama, 2026], which is this paper's question reached independently. The agreement runs deeper: a rule there returns one of five statuses, and two of them, `unknown` and `not-applicable`, are excluded from the score rather than counted as passes. The unit differs, and so does where an outcome comes from. adduce grades a repository and a rule author selects among five flat statuses, where one assertion here binds one printed value to one declared address, no score is computed, and the outcome follows from which of the three stages stopped. The two compose: adduce runs `repro verify` as one of its rules.

Artifact-level verification. Code Ocean verifies a capsule before publication and states that the check establishes mechanical reproducibility, a user pressing run and obtaining results, and separately that it does not check whether a capsule's results match the article it accompanies [Code Ocean, 2026]. Whole Tale packages code, data, methods and environment with the narrative as a re-instantiable tale [Brinckman et al., 2019]. In both, the unit of citation and the unit of verification are the same object, so the outcome is one property of the whole. Here the unit of verification is finer than the unit of citation, and a report states which of a manuscript's values an artifact still supports rather than whether the artifact runs.

Re-execution. Workflow systems rerun the steps whose inputs changed [Iterative, Inc., 2020, Köster and Rahmann, 2012], and a codechecker regenerates some or all of a paper's results [Nüst and Eglen, 2021]. ScientistOne ties each numeric sentence of a paper it writes to a line of its experiment log, and a language model reads the score its audit checks [Meng et al., 2026]. `repro reproduce` compares each printed number with the value at its declared address in the rerun output, and reports a rerun that did not happen apart from a rerun that disagreed (Appendix K).

Entailment, markup and write-time binding. FEVER, SciFact and CliniFact predict a label given retrieved text and score it against human annotation [Thorne et al., 2018, Wadden et al., 2020, Zhang et al., 2025], and ClaimReview binds claim text to the work it appeared in while recording no digest and enumerating no outcomes [Schema.org, 2026]. Our outcomes name what a byte or decimal comparison found. Literate programming generates the document from the execution producing its values [Leisch, 2002], which removes the drift, requires the manuscript to be authored inside the toolchain, and absorbs a change where this reports one.

Abstention as an outcome. Fellegi and Sunter classify a candidate pair as match, non-match, or possible match held for review, and prove that rule minimizes the review region at fixed error bounds [Fellegi and Sunter, 1969]. The three-way structure, with abstention as a first-class outcome rather than a failure, is the shape Table 3 takes. Their third state is reached probabilistically from estimated agreement patterns; ours is reached because an extractor was absent or a pointer did not resolve.

Verification in other fields. Metrology defines verification as evidence that specified requirements are met, and keeps verification apart from validation, which asks whether those requirements suit an intended use [Joint Committee for Guides in Metrology, 2012]; Reproducible Science performs verification. The philosophy of measurement separates an instrument's reading from the result inferred from the reading [Tal, 2017], and an extractor is an instrument in this sense: in Section 4.2, six extractors read the same bytes and returned six results. In formal verification, many early automated theorem provers performed proofs without recording them, so a result could not be checked afterwards [MacKenzie, 2001]; here every verdict is stored with its pinned artifacts.

7 Limitations

The extractor is trusted. Content addressing establishes that an artifact's bytes did not change and nothing about whether the extractor read them correctly. A layout parser that splices a value from the wrong column, or an optical pass that reads `0.9489` as `0.9439`, leaves the digest intact and returns a confident answer. The blockchain literature calls this the oracle problem [Albizri and Appelbaum, 2021]. Recording the extractor's identity and a digest of the text it produced makes a changed reading visible, which addresses drift and not correctness.

Registration establishes order, not honesty. A run record states when a plan was registered and when the run began, and the check establishes that those two are consistent. It does not establish that the registration is contemporaneous with what it claims. Each freeze requests a date from outside the repository [Adams et al., 2001]: its digest goes to OpenTimestamps [Todd, 2016] and the returned proof fixes the latest date the plan could have been written (Appendix G). A timestamp cannot show that a plan was not written earlier and withheld.

Declaration is authored, and coverage is gameable. Only declared numbers are checked, so a project can pass by declaring only the numbers it expects to match. We state coverage alongside outcomes so that a reader can see what fraction of the manuscript's numbers was checked.

8 Conclusion

Re-running an analysis establishes that the code runs. Verification debt accumulates in the difference between that and what a manuscript says it produced. Values, quotations and registered plans differ in how each is extracted and little else, so one engine reports all three. The three stages exist because disagreement, absence and non-execution have different remedies, and because collapsing them produced a run that examined nothing.

References

Carlisle Adams, Pat Cain, Denis Pinkas, and Robert Zuccherato. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). Request for Comments 3161, Internet Engineering Task Force, 2001. URL https://www.rfc-editor.org/info/rfc3161.

Abdullah Albizri and Deniz Appelbaum. Trust but Verify: The Oracle Paradox of Blockchain Smart Contracts. *Journal of Information Systems*, 35(2):1–16, 2021. doi: 10.2308/isys-19-024. URL https://doi.org/10.2308/isys-19-024.

Association for Computing Machinery. Artifact Review and Badging, Version 1.1. ACM publications policy, August 24, 2020, 2020. URL https://www.acm.org/publications/policies/artifact-review-and-badging-current.

Adam Brinckman, Kyle Chard, Niall Gaffney, Mihael Hategan, Matthew B. Jones, Kacper Kowalik, Sivakumar Kulasekaran, Bertram Ludäscher, Bryce D. Mecum, Jarek Nabrzyski, Victoria Stodden, Ian J. Taylor, Matthew J. Turk, and Kandace Turner. Computing environments for reproducibility: Capturing the "Whole Tale". *Future Generation Computer Systems*, 94:854–867, 2019. doi: 10.1016/j.future.2017.12.029. URL https://doi.org/10.1016/j.future.2017.12.029.

Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. ReproZip. In *Proceedings of the 2016 International Conference on Management of Data*, pages 2085–2088. ACM, 2016. doi: 10.1145/2882903.2899401. URL https://doi.org/10.1145/2882903.2899401.

Harshil Chudasama. adduce: a local research-artifact auditor, 2026. URL https://github.com/QHarshil/adduce. Software, version 0.1.2, released 2026-08-04. Author and title taken from the project's own CITATION.cff.

Code Ocean. Code Ocean's verification process for computational reproducibility & quality, 2026. URL https://docs.codeocean.com/osl-guide/publishing-on-code-ocean/the-verification-process/code-oceans-verification-process-for-computational-reproducibility-and-quality. Open Science Library documentation. Read 2026-08-27.

Manuel Corpas. ClawBio: Bioinformatics-Native AI Agent Skill Library. Software, version 0.5.2; reproducibility guide at commit 5ca937c, 2026. URL https://github.com/ClawBio/ClawBio.

Crossref and Retraction Watch. Retraction watch data, 2026. URL https://gitlab.com/crossref/retraction-watch-data.

Tianyu Ding, Aditya Nannapaneni, Bingfan Liu, and Ling Zhang. Autonomous Research Agents: A Survey of AI Scientists and the Verification Gap, 2026. URL https://arxiv.org/abs/2608.05179.

Ivan P. Fellegi and Alan B. Sunter. A Theory for Record Linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969. doi: 10.1080/01621459.1969.10501049. URL https://doi.org/10.1080/01621459.1969.10501049.

Generative Expert Labs. Paperclip: Agent-native indexing of scientific literature and data, 2026. URL https://paperclip.gxl.ai/. Software. CLI client at https://github.com/GXL-ai/paperclip (Apache-2.0).

Sebastiaan P. Huber, Spyros Zoupanos, Martin Uhrin, Leopold Talirz, Leonid Kahle, Rico Häuselmann, Dominik Gresch, Tiziano Müller, Aliaksandr V. Yakutovich, Casper W. Andersen, Francisco F. Ramirez, Carl S. Adorf, Fernando Gargiulo, Snehal Kumbhar, Elsa Passaro, Conrad Johnston, Andrius Merkys, Andrea Cepellotti, Nicolas Mounet, Nicola Marzari, Boris Kozinsky, and Giovanni Pizzi. AiiDA 1.0, a scalable computational infrastructure for automated reproducible workflows and data provenance. *Scientific Data*, 2020. doi: 10.1038/s41597-020-00638-4. URL https://doi.org/10.1038/s41597-020-00638-4.

Iterative, Inc. DVC: Data Version Control, 2020. URL https://dvc.org/. Software.

Joint Committee for Guides in Metrology. International Vocabulary of Metrology – Basic and General Concepts and Associated Terms (VIM). JCGM 200:2012, 3rd edition, 2012. URL https://www.bipm.org/documents/20126/2071204/JCGM_200_2012.pdf.

Project Jupyter, Matthias Bussonnier, Jessica Forde, Jeremy Freeman, Brian Granger, Tim Head, Chris Holdgraf, Kyle Kelley, Gladys Nalvarte, Andrew Osheroff, M Pacer, Yuvi Panda, Fernando Perez, Benjamin Ragan-Kelley, and Carol Willing. Binder 2.0 - Reproducible, interactive, sharable environments for science at scale. In *Proceedings of the Python in Science Conference*, pages 113–120. SciPy, 2018. doi: 10.25080/majora-4af1f417-011. URL https://doi.org/10.25080/Majora-4af1f417-011.

Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012. doi: 10.1093/bioinformatics/bts480. URL https://doi.org/10.1093/bioinformatics/bts480.

Friedrich Leisch. Sweave: Dynamic Generation of Statistical Reports Using Literate Data Analysis. In *Compstat: Proceedings in Computational Statistics*, pages 575–580. 2002. doi: 10.1007/978-3-642-57489-4_89. URL https://doi.org/10.1007/978-3-642-57489-4_89.

Simone Leo, Michael R. Crusoe, Laura Rodríguez-Navas, Raül Sirvent, Alexander Kanitz, Paul De Geest, Rudolf Wittner, Luca Pireddu, Daniel Garijo, José M. Fernández, Iacopo Colonnelli, Matej Gallo, Tazro Ohta, Hirotaka Suetake, Salvador Capella-Gutierrez, Renske de Wit, Bruno P. Kinoshita, and Stian Soiland-Reyes. Recording provenance of workflow runs with RO-Crate. *PLOS ONE*, 2024. doi: 10.1371/journal.pone.0309210. URL https://doi.org/10.1371/journal.pone.0309210.

Shicheng Liu, Krista Opsahl-Ong, Andrew Drozdov, Julian Killingback, Arnav Singhvi, Ronak Pradeep, Michael Bendersky, James C. Dickerson, and Monica S. Lam. AI-driven reproducibility and analytical robustness in scientific discovery. Preprint, 2026. URL https://drive.google.com/file/d/1RXe-z076bd3jKqP0LkzAto8cMvce0soZ/view.

Rodrigo Luger, Megan Bedell, Daniel Foreman-Mackey, Ian J. M. Crossfield, Lily L. Zhao, and David W. Hogg. Mapping stellar surfaces III: An Efficient, Scalable, and Open-Source Doppler Imaging Model, 2021. URL https://arxiv.org/abs/2110.06271.

Donald MacKenzie. *Mechanizing Proof: Computing, Risk, and Trust*. The MIT Press, 2001. doi: 10.7551/mitpress/4529.001.0001. URL https://doi.org/10.7551/mitpress/4529.001.0001.

Rui Meng, Bhavana Dalvi Mishra, Jiefeng Chen, Chun-Liang Li, Palash Goyal, Mihir Parmar, Yiwen Song, Yale Song, Rajarishi Sinha, Parthasarathy Ranganathan, Burak Gokturk, Jinsung Yoon, and Tomas Pfister. ScientistOne: Towards Human-Level Autonomous Research via Chain-of-Evidence, 2026. URL https://arxiv.org/abs/2605.26340.

Jaehyun Nam, Jinsung Yoon, Yanzhou Pan, Yubo Wang, Rui Meng, Parthasarathy Ranganathan, and Tomas Pfister. ScientistTwo: Pioneering the Human Knowledge Frontier with Autonomous AI, 2026. URL https://arxiv.org/abs/2609.19644.

Daniel Nüst and Stephen J. Eglen. CODECHECK: an Open Science initiative for the independent execution of computations underlying research articles during peer review to improve reproducibility. *F1000Research*, 10:253, 2021. doi: 10.12688/f1000research.51738.2.

Open Source Security Foundation. SLSA: Supply-chain Levels for Software Artifacts, 2023. URL https://slsa.dev/. Specification.

João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, pages 507–517, 2019. doi: 10.1109/MSR.2019.00077. URL https://doi.org/10.1109/MSR.2019.00077.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program), 2020. URL https://arxiv.org/abs/2003.12206.

Retraction Watch. Top 10 most highly cited retracted papers. The Retraction Watch Leaderboard, 2026. URL https://retractionwatch.com/the-retraction-watch-leaderboard/top-10-most-highly-cited-retracted-papers/.

Sheeba Samuel, Daniel Mietchen, Jungsan Kim, Waqas Ahmed, and Martin Gaedke. ReproScore: Separating Readiness from Outcome in Research Software Reproducibility Assessment, 2026. URL https://arxiv.org/abs/2605.13275.

Schema.org. ClaimReview and Claim, 2026. URL https://schema.org/ClaimReview. Vocabulary types ClaimReview and Claim, release 30.0 of 2026-03-19; definitions checked against the released JSON-LD artifact. Read 2026-08-27.

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark, 2024. URL https://arxiv.org/abs/2409.11363.

Eran Tal. Calibration: Modelling the measurement process. *Studies in History and Philosophy of Science Part A*, 65-66:33–45, 2017. doi: 10.1016/j.shpsa.2017.09.001. URL https://doi.org/10.1016/j.shpsa.2017.09.001.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018. doi: 10.18653/v1/n18-1074. URL https://doi.org/10.18653/v1/n18-1074.

Peter Todd. OpenTimestamps: Scalable, Trust-Minimized, Distributed Timestamping with Bitcoin. Blog post, September 2016. URL https://petertodd.org/2016/opentimestamps-announcement.

Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

Elliot Tower. Diagnosing drug failure modes from cross-design evidence across ten disease domains, 2026a. URL https://doi.org/10.5281/zenodo.21227353.

Elliot Tower. Higher-order knockout epistasis is generated by dynamics, not molecular rules, in 28 Boolean gene regulatory networks, 2026b. URL https://doi.org/10.5281/zenodo.22653498.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or Fiction: Verifying Scientific Claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. doi: 10.18653/v1/2020.emnlp-main.609. URL https://doi.org/10.18653/v1/2020.emnlp-main.609.

Boya Zhang, Alban Bornet, Anthony Yazdani, Philipp Khlebnikov, Marija Milutinovic, Hossein Rouhizadeh, Poorya Amini, and Douglas Teodoro. A dataset for evaluating clinical research claims in large language models. *Scientific Data*, 2025. doi: 10.1038/s41597-025-04417-x. URL https://doi.org/10.1038/s41597-025-04417-x.

A Retractions, errors and misconduct in published results

Reproducible Science checks that a manuscript agrees with its artifacts, which covers some of the reasons published results are retracted and not others. To determine which reasons the checks cover, we examine 34 cases of retraction, correction, or misconduct finding, and classify each by the cause stated in its primary record.

Detected failure modes. Table 8 gives the coverage of each failure mode, counting each case once under its principal cause. The checks cover a stated problem in 14 of the 34 cases: the principal cause in 8 (6 detected and 2 reported as unchecked), and missing data reported alongside the principal cause in 6 others. The checks detect three failure modes directly: (1) a printed value that disagrees with the authors' own records fails the check of that number against its results file, (2) a report that departs from its registered plan differs from the frozen and timestamped plan, and (3) a reference that does not exist cannot be pinned to a source.

Table 8: Failure modes among the 34 cases and the coverage of each by Reproducible Science. Each case is counted once, under its principal cause. †Added after the records were read.

Failure mode	Cases	Coverage
printed value disagrees with the authors' own records	3	**detected**: the printed number fails the check against its results file
report departs from the registered plan	1	**detected**: the report differs from the frozen, timestamped plan
references that do not exist	2	**detected**: a reference that does not exist cannot be pinned to a source
data could not be produced for audit	2	**reported as unchecked**: the sealed digest records that the file existed
error inside the analysis	9	**not detected**: the sealed script and inputs allow the analysis to be rerun
data fabricated or falsified	8	**not detected** if fabricated before sealing; a later change to a sealed file is detected
image manipulation or duplication	4	**not addressed**
study conducted differently from its description†	4	**not addressed**
none of the above	1	**not addressed**
Total	34	

Undetected failure modes. Data that cannot be produced for an audit are reported as unchecked rather than verified. An error inside the analysis passes every check, because the manuscript agrees with the output of the faulty analysis, but the sealed script and inputs allow a reader to rerun the analysis and locate the error. Reproducible Science does not address data fabricated before sealing, manipulated images, or a study conducted differently from its published description.

Case selection. The primary record of a case is its retraction notice, a finding of the United States Office of Research Integrity, or a published reanalysis. The cases comprise nine of the ten most-cited retracted papers listed by Retraction Watch [Retraction Watch, 2026], publications named in Office of Research Integrity findings, retractions drawn from the Retraction Watch dataset [Crossref and Retraction Watch, 2026] by reason tag, and well-known examples such as the Duke chemotherapy predictors and the Surgisphere studies. We select the cases as examples of each failure mode. We define ten failure modes before reading any record and add an eleventh for four records that fit none of the ten.

Retraction Watch dataset. To place the cases in context, we analyze the reasons recorded in the Retraction Watch dataset, which covers 67,530 retractions. We exclude the 24,166 retractions attributed to paper mills or compromised peer review, because these concern the publication process, and group the reason tags of the remaining 43,364 (Table 9). Among these, 4,273 retractions (9.8%) cite an error in the analysis, data, methods, results or text, and 1,909 (4.4%) cite original data or

images that were not provided. Retractions that cite unavailable data or images follow publication by a median of 4.91 years, compared with 1.31 years for retractions that cite an error.

Table 9: Retractions in the Retraction Watch dataset by group of reason tags, among the 43,364 not tagged for paper mills or compromised peer review. A retraction can fall in several groups. Years run from the paper's date to the retraction's date.

Reason for retraction	Retractions	Share	Median years to retraction
image duplication, manipulation, fabrication or concerns	7,171	16.5%	4.39
error stated in analysis, data, methods, results or text	4,273	9.8%	1.31
original data or images not provided	1,909	4.4%	4.91
falsification or fabrication of data	1,659	3.8%	4.27
results not reproducible	861	2.0%	2.01
limited or no information in the notice	11,957	27.6%	0.11

Cases. Table 10 gives the cause quoted from the primary record of each case.

Table 10: Thirty-four cases, grouped by failure mode, with the cause in the words of the primary record: a retraction notice, a finding of the Office of Research Integrity, or a published reanalysis.

Case	What its primary record says
Printed value disagrees with the authors' own records	
Abegaz et al., PLOS ONE, 2014	"the estimated number of deaths and injuries reported cannot be reproduced when using the unmatched and matched police and hospital data"
Sun et al., PLOS ONE, 2024	"the p50 median value is lower than the reported minimum value" Also states that data, images or records could not be produced.
Wansink et al., Archives of Pediatrics and Adolescent Medicine, 2012	"We had erroneously reported the age group as children ranging from 8 to 11 years old; however, the children were 3 to 5 years old."
Report departs from the registered plan	
67 trials in five medical journals, 2015 to 2016 (a cohort, not one publication)	"365 novel outcomes were reported without declaration in total, a mean of 5.4 per trial"
References that do not exist	
Khalilzadeh et al., PLOS ONE, 2026	"Citation of unretrievable references 1, 2, 4, 5, 8, 10, 12–18, 20–22, 33, 36, 51, 61, 75, and 79."
Mohammad et al., PLOS ONE, 2026	"Citation of unretrievable References 2, 3, 6, 7, 8, 18, 20, and 26."
Data could not be produced for audit	
Mehra et al., Lancet, 2020	"Surgisphere would not transfer the full dataset, client contracts, and the full ISO audit report to their servers for analysis"
Mehra et al., New England Journal of Medicine, 2020	"the raw data could not be made available to a third-party auditor, we are unable to validate the primary data sources"
Error inside the analysis	
Morisky et al., Journal of Clinical Hypertension, 2008	an independent statistical review "concluded that the results were misleading due to issues regarding the sensitivity and specificity of the medical adherence scale used"
Gebrechorkos et al., Nature, 2025	"area-weighted grid cells were not used when calculating study-wide averages, an error that affected numerical values in the main text"
Islam et al., PLOS ONE, 2019	"an error in the p-value calculation caused by a coding error"
Zhang et al., PLOS ONE, 2025	a formula "contains an incorrect minus sign, which should be a plus sign"
Okray et al., Nature, 2023	the authors found "errors in the pipeline originally used for data analysis"
Hewitson et al., PLOS ONE, 2021	the correlation analysis "included typically developing (TD) participants in error"

Case	What its primary record says
Rahman et al., PLOS ONE, 2021	"the PSS mean reported in [1] is consistent with incorrect coding of these items" Also states that data, images or records could not be produced.
Huang et al., Lancet, 2021	an expression of concern was published "because of the substantive data correction request from the authors following comments from a reader"
Reinhart and Rogoff, American Economic Review, 2010	"coding errors, selective exclusion of available data, and unconventional weighting of summary statistics lead to serious errors"
Data fabricated or falsified	
Potti et al., Nature Medicine, 2006	"Respondent altered data sets to improve the accuracy of predictors for response to treatments"; "the results incorporate several simple errors that may be putting patients at risk"; "a corruption of several validation data sets precludes conclusions regarding these signatures"
Potts Kant et al., Pediatric Research, 2013	falsified data by "changing numerical values in the datasets generated by the apparatus"
Savine and Braver, Cognitive, Affective and Behavioral Neuroscience, 2012	"Respondent exaggerated (1) the effect of incentive context on response times and error rates in Table 1"
Skau et al., Cell, 2016	falsification by "selectively reporting by inappropriate inclusion/omission or alteration of data points in ten (10) figures"
Hauser et al., Cognition, 2002	"Respondent published fabricated data in Figure 2 of the paper"
Linn et al., Cellular and Molecular Biology, 2003	"he falsified the values in Tables 2-5 by altering the values that he had obtained from another author's manuscript"
Thiruchelvam et al. (publication to be identified from the finding)	misconduct by "falsifying and fabricating cell count data that she claimed to have obtained through stereological methods"
Taylor and Gercel-Taylor, Gynecologic Oncology, 2008	two investigations "found evidence of falsification, fabrication, and plagiarism of data"
Image manipulation or duplication	
Jiang et al., Nature, 2002	"The original images for Figures 6a, 6e and 6m could not be retrieved by the authors"
Lesné et al., Nature, 2006	the figures "show signs of excessive manipulation, including splicing, duplication and the use of an eraser tool. The data cannot be verified from the records."
Zou et al., Nature, 2001	falsified figures by "manipulating the images to alter the number and location of positively stained cells"
Liu et al., PLOS ONE, 2014	"the data underlying the panels of concern are no longer available"
Study conducted differently from its description	
Wakefield et al., Lancet, 1998	the paper's claims "that children were 'consecutively referred' and that investigations were 'approved' by the local ethics committee have been proven to be false"
Estruch et al., New England Journal of Medicine, 2013	"Because of irregularities in the randomization procedures, we wish to retract the following article"
Gautret et al., International Journal of Antimicrobial Agents, 2020	"The journal has not been able to establish whether all patients could have entered into the study in time" Also states that data, images or records could not be produced.
LaCour and Green, Science, 2014	"Survey incentives were misrepresented."; "LaCour has not produced the original survey data from which someone else could independently confirm the validity of the reported findings"
None of the above	
Fukuhara et al., Science, 2005	"we have found that not all preparations of visfatin bind to and activate the insulin receptor"

Limitations. The 34 cases are illustrations, at least one per failure mode. Table 9 reports how often each reason for retraction occurs among 43,364 retractions in the Retraction Watch dataset. The reason tags reflect the dataset curators' reading of each notice, and 27.6% of the retractions analyzed are tagged as having a notice with limited or no information. In the full dataset, 1,725 of the 2,464 retractions tagged for unavailable data or images also have an image-related tag, so most concern missing original images. The analysis shows which failure modes the records of

Reproducible Science would expose; it does not show that any of these cases would have been corrected sooner.

B Terms

Table 11 defines the terms used in this paper.

Table 11: Terms as this paper uses them.

Term	Meaning
Reproducible Science	the `reproducible-science` package (containing four commands: `prereg`, `results`, `citations` and `repro`) and the optional Claude Code plugin and mod
engine	the code in `repro` that resolves pins, evaluates each assertion and reports the three stages
artifact	a file a claim rests on, pinned by its SHA-256 digest
claim	a sentence in the manuscript that states a number or a quotation
evidence assertion	one check bound to a claim: an artifact, a location in it, and the value or passage expected there
manifest	the file, `repro.yaml`, listing a project's artifacts, claims and assertions
claim record	a file pinning the quotations taken from one source, read by `citations`
plan	the preregistration document
freeze	the record of a plan's digest, commit and time, after which the plan does not change
ledger	the append-only record of sealed inputs, runs and the claims bound to them, kept by `results`
declaration	a frozen plan, a sealed input, a recorded run, a pinned quotation or an assertion: what a project writes for the commands to read

C Conformance suite and fault injection

Conformance. The suite holds 25 fixtures: five addressing a table cell, four comparing a value, three each for prose anchors, two-sided correspondences and artifact availability, two each for a pointer and a passage, and one each for an unpinned artifact, a broken pin, and a claim offering no evidence. Each records its canonical expected output as an (outcome, reason, artifact validity) triple, so a case cannot pass by reaching the right verdict for the wrong cause. Assertions over them include that an absent pointer yields `extraction=absent` and never `comparison=mismatch`, and that an injected `TypeError` yields `error` and never `unchecked`.

Fault injection. Four faults are injected and the reported outcome recorded: a results file edited after drafting yields a broken pin with decisions against it marked non-authoritative; an absent extraction toolchain yields `unchecked` with reason `extractor_missing`; a registered plan edited after registration yields a broken pin on the plan artifact; a backend raising `TypeError` yields `error` with reason `backend_defect`. Each distinguishes a class the flattened view would otherwise merge.

The quote variant was run over an audit of sixteen published claims: 2,512 quotations against 125 content-addressed sources, of which the fifteen PDFs of the papers under audit carry 2,364. Every source carries a digest, so the document a failing quotation was checked against is byte-identical to the one the manuscript cited. Under poppler's layout mode, 113 quotations did not resolve. One rater, who also wrote the quotations, attributed every one to extraction rather than to misquotation, each cause being recoverable from the recorded extractor output: 110 came from the mode's preservation of page geometry, which severs every sentence crossing the gutter of a two-column paper, 2 from a text layer omitting a character the passage contains, and 1 from a hyphen at a line end that de-hyphenation cannot attribute. On this corpus a two-state checker therefore carries a false-accusation rate of 4.5%: it would have recorded all 113 as misquotations, each an accusation against the manuscript, and the corpus held no misquotation for it to find.

Table 12: Passages resolved per extractor over one corpus, and the cost of each. The two poppler modes fail in opposite directions and together reach 2,360. Ranking by total resolved is a poor guide to which extractors to consult: the low-level parser resolves 598 fewer passages than the best single extractor and is the only one that extends the union, reaching 2,362. Two quotations resolve under no extractor.

Extractor	Resolved of 2,364	Seconds
poppler, reading order	2,314	6.9
poppler, layout	2,250	6.9
pure-Python reader	2,229	43.9
rendering-engine binding	2,092	4.8
structural extractor	1,973	6.2
low-level parser	1,716	2.7
both poppler modes	2,360	13.8
all six	2,362	71.4

Table 13: Policy profiles over identical facts. Every profile requires that at least one assertion was evaluated, because a project with no evidence anywhere otherwise satisfies every condition trivially. `not_offered` marks a claim that offers no evidence.

Profile	mismatch	not_found	unchecked	not_offered	unpinned
exploratory	error	warning	warning	ignore	ignore
publication	error	error	warning	warning	warning
publication, confirmatory claims	error	error	**error**	**error**	—
strict	error	error	error	error	error

D Session status

The Claude Code mod reads the project's records at the end of each turn and draws them above the prompt, one row per command: the plans frozen out of the plans present, the runs sealed and the claims bound to them, the quotations pinned, and the manifest's checks verified. A row for a record the project lacks names the command that creates it. A line appears under the prompt only when a record no longer matches.



Figure 2: The state of a project's records as a session shows it. One plan is frozen, one run is sealed with one claim bound to it, no quotation is pinned, and three of the manifest's four checks verify. The line under the prompt reports the fourth: a number in the manuscript no longer matches its results file.

E Timing

Each time in Section 3 is the median of 10 runs of the command a user types, each in a new process, on a 2019 16-inch MacBook Pro (`MacBookPro16,1`: 6-core Intel Core i7-9750H at 2.6 GHz, 16 GB of memory, macOS 26.5, Python 3.13.12), with version 0.5.2. `repro verify` over this paper's manifest ranges from 0.53 to 0.66 seconds, and `citations verify` over the quotations this paper pins from 3.92 to 4.88 when every run reads every source. By default `citations verify` keeps the text extracted from a source whose bytes have not changed, and then takes 1.47 seconds.

Table 14 gives `repro verify` over generated projects of increasing size, beside `sha256sum -c` over the same artifacts, the least any check of pinned files can cost. Each project holds a text, a table and a manifest of correspondence assertions, each tying a number in the text to a cell of the table. The command starts in 0.192 seconds, which is most of its cost up to 100 assertions. Artifact size adds what hashing adds: over 4 GB the check takes 2.416 seconds and hashing alone takes 2.163. Time grows faster than the number of assertions past 10,000, because each text anchor is searched for across the whole text.

Table 14: Seconds to verify a generated project, by the number of assertions and by the bytes of the artifacts they are checked against. Each time is the median of 5 runs, each in a new process, of version 0.5.2 on a GitHub-hosted runner (`ubuntu-24.04`: AMD EPYC 9V45, 2 processors, 7 GB of memory, Python 3.12.15). The last four rows hold the assertions at 100 and add pinned files that no assertion reads.

Assertions	**Artifact bytes**	`repro verify`	`sha256sum -c`
10	647	0.213	0.001
100	6,389	0.238	0.001
1,000	63,807	0.556	0.001
10,000	638,009	6.733	0.002
100,000	6,380,011	193.128	0.005
100	1,054,965	0.223	0.002
100	16,783,605	0.246	0.010
100	268,441,845	0.372	0.142
100	4,294,973,685	2.416	2.163

F Certified reproductions

CODECHECK issues a certificate when an independent codechecker regenerates a paper's outputs, and each certificate's manifest names the regenerated files and the figure or table each belongs to [Nüst and Eglen, 2021]. The correspondence between file and table is therefore written by a third party. We run `repro verify` over the public register to measure how much of a certified reproduction can be checked number by number. The plan is registered before any output file is opened.

Result. Six certificates link fifteen tables to regenerated files, and 338 of the 713 numbers those tables print are bound to a cell of a file. In 326 of the 338, the paper and the certificate's file hold the same number. Twelve differ, in two certificates, and each difference has a cause that can be read from the files (Table 18). The counts describe what can be checked in certified reproductions and do not estimate how often published numbers are wrong.

From the register to checkable tables. Table 15 follows the register down to the certificates Reproducible Science can use. The manifests name 545 outputs, of which 419 are figures. Six certificates name a file Reproducible Science reads, keep it in their repository, and say which table of the paper it holds.

Table 15: Certificates in the CODECHECK register that reach each condition. Three certificates describe their files without naming a table, and four name a file their repository does not hold under that name.

condition	certificates
in the register	133
names an output in a format read as values, not marked as outside the manuscript	13
that output is in the certificate's repository	9
its manifest comment names a table the paper has	6

Coverage. A number is bound when it has one address in the paper's extracted text and one cell in the regenerated file. The bindings are written with Claude Code (Anthropic) from views of each output in which every value is masked, and are locked before any comparison runs. Four certificates

bind between 64 of 92 and 137 of 140 numbers (Table 16). Two bind none: their outputs hold formatted text, in one a median with its interval and units in each cell and in the other a single column named `text`. An output that reproduces a table's appearance has no cell to compare a number against.

Table 16: Numbers printed in linked tables, counting at most 60 per table, numbers bound on both sides, and bound numbers that are the same in the paper and in the certificate's file.

certificate	printed	bound	same number	differ
2025-018	140	137	133	4
2020-025	90	86	86	0
2022-001	92	64	56	8
2024-020	58	51	51	0
2020-011	33	0	—	—
2020-008	300	0	—	—
all	713	338	326	12

Agreement. Reproducible Science reports 254 of the 338 bound numbers as agreeing at the precision the paper prints. It reports 69 as disagreeing and does not read 15, and 72 of those 84 are the same number (Table 17). The registered rule reads each output at the last commit before the certificate's check time. Sixteen of 37 linked outputs exist there, so each output is read at the first commit holding it, a change logged before any output is opened. The 30 numbers of one table of certificate 2020-025 differ from that first revision of its output file and equal the revision at the check time in every cell; the certificate does not record which revision was certified. The 15 unread numbers, 11 of them equal to the file, expose a defect in Reproducible Science, whose numeric parser rejects a thousands separator.

Table 17: The 338 bound numbers, by how the paper's numeral compares with the certificate's file.

comparison	numbers
the check reports agreement	254
the paper truncates where the check rounds	31
equal at the output file's revision at the check time	30
equal once a thousands separator is removed	11
differ	12
all	338

Differences. Table 18 lists the 12 numbers that differ. Five are standard deviations that equal the file's value multiplied by $\sqrt{(n-1)/n}$ with $n = 1{,}070$, which is consistent with a paper that divides by n where the regenerated file divides by $n-1$; the same factor reproduces all 12 standard deviations counted in that table. One regression coefficient is printed without its exponent. Three are memory measurements, 0.1% to 6.5% from the file's value.

Planted changes. Agreement means little if the check cannot report a disagreement, so each bound number is checked again on copies carrying one planted change (Table 19). The check reports a one-unit change in the last printed digit of the file in all 254 cases and of the paper in 240. In ten of the fourteen misses the file stores fewer digits than the paper prints, and in four the changed numeral sits among identical neighbors that serve as its anchors. No deleted row, column or file is reported as agreement or as a disagreement.

Table 18: The 12 bound numbers that differ between the paper and the certificate's file. File values are shortened to the digits needed to show the difference.

certificate	quantity	paper	file	difference
2022-001	standard deviation	30.33	30.3485	divisor n or $n-1$
2022-001	standard deviation	21.80	21.8124	divisor n or $n-1$
2022-001	standard deviation	28.20	28.2116	divisor n or $n-1$
2022-001	standard deviation	84,154.56	84,193.92	divisor n or $n-1$
2022-001	standard deviation	3,132.90	3,134.36	divisor n or $n-1$
2022-001	regression coefficient	5.273	5.273×10^{-7}	exponent not printed
2022-001	median	4,984.44	4,984.4349	last printed digit
2022-001	maximum	827,067.40	827,067.43	0.03
2025-018	run time, seconds	12.659	12.658475	last printed digit
2025-018	memory, megabytes	3.1	2.897	6.5%
2025-018	memory, megabytes	610.8	611.443	0.1%
2025-018	memory, megabytes	2310.8	2329.968	0.8%

Table 19: Planted changes on copies of the bound numbers, and how many the check reports with the outcome and reason its specification gives. The first three rows use the 254 numbers that agree; the last three use all 338.

planted change	expected report	as expected
none	agreement	254 of 254
last printed digit of the file's value	disagreement	254 of 254
last printed digit of the paper's value	disagreement	240 of 254
addressed row deleted	row absent	338 of 338
addressed column deleted	column absent	338 of 338
output file deleted	not checked, file missing	338 of 338

G Outside timestamps

`prereg freeze` takes one SHA-256 digest over the plan and sends that digest, and nothing else, to the OpenTimestamps calendar servers [Todd, 2016]. The proof they return is kept beside the plan and committed with it. A proof is pending until the calendars commit it into a Bitcoin block, whose date neither the author nor a calendar operator can change, usually within hours; `prereg timestamp` then completes it and reports the block and its date, the latest date at which the plan could have been written. `prereg check` reads the proof without the network and fails when the proof is of a digest other than the frozen one. No option skips the request: a freeze made with no network succeeds and reports the timestamp as owed on every check until it is made. `results timestamp` dates the head of the ledger the same way. The date is then held by a public record the author cannot rewrite, which a repository's own history is not.

H Reproducible Science Audits

This appendix audits 3 preregistered studies conducted with Reproducible Science: two under journal review and one in progress. Each audit gives a description of the study, an example finding, what each command found in its repository, and the findings made while it was written. Table 20 gives the result from a clone for the two under review, and Table 21 their findings.

Table 20: What Reproducible Science established from a clone of each project, at the pinned commit.

project	appendix	registered plans	quotations	ledger
Cross-design evidence discordance	I	4 of 6 registrations unchanged, 1 appended, 1 changed	175 of 175 found	not in the repository
Knockout epistasis	J	3 of 3 plans unchanged	18 declared, sources absent	chain intact, 153 events

Findings are coded from each project's records: commits, ledger events, plan logs, claim files and session transcripts. A finding counts when a command, a script that performed the same check before the command existed, a hook, or an act Reproducible Science requires (sealing an input, binding a number, pinning a quotation) exposed it and a record says so. The coding rules were fixed before the records were read. Defects found by a person or raised in review are not counted as findings.

Table 21: Findings, by project and severity. A *claim* finding would change a reported number or conclusion. A *source* finding is a value or reference attributed to a work that does not contain it. A *record* finding is a break in provenance that leaves every reported number correct.

project	appendix	claim	source	record	cosmetic	total
Cross-design evidence discordance	I	5	14	8	4	31
Knockout epistasis	J	3	2	10	3	18
total		8	16	18	7	49

I Audit: Cross-design evidence discordance

Description. A preregistered study of drug-trial outcomes [Tower, 2026a].[4] The clone carries 365 tracked files, 6 registration documents, 94 claim records, 175 quotations and 75 bibliography entries.

All Findings. 31: 5 claim, 14 source, 8 record, 4 cosmetic.

Example Finding. `results verify` found two recorded files edited since they were recorded and one that had moved.

Reviews. Of the 31 findings: 0 before submission, 11 review round 1, 2 review round 2, 0 review round 3, 17 review round 4, 1 after round 4.

Table 22: Cross-design evidence discordance: what each command found in a clone at the pinned commit, and what it could not establish. Exit 0 is a pass, 1 a finding, and 2 nothing to read.

command	exit	what it found	what it could not establish
`prereg`	1	6 registration documents, each frozen by a commit recorded in the document: 4 unchanged, 1 with text added after the frozen text, and 1 changed after its freeze	whether a registration was written before the outcomes it governs were seen
`results`	2	no ledger	which run any reported number came from
`citations`	0	175 of 175 quotations found in their sources; 67 of the 94 sources were read from the library and matched to the SHA-256 the record pins	whether a quotation supports the sentence that cites it
`citations audit`	0	75 bibliography entries, 74 carrying an identifier; 64 agree with the record the identifier resolves to and 10 disagree	whether the entry with no identifier is correct
`repro`	2	no evidence manifest	any assertion, none having been declared

Table 23: Reviewer comments (4 reviewers, 6 reports) and findings for the cross-design study, by kind.

Reviewer comments	Count
design and argument	20
reporting and presentation	12
statistical method	10
references	1
code and data availability	0
other	2
Total	45

Findings	Count
value or citation absent from the cited source	9
value differing from its source	6
wrong reference metadata	11
number bound to no run	2
other record findings	3
Total	31

[4] https://github.com/elliottower/cross-design-evidence-discordance, commit `c6a7a06`.

J Audit: Knockout epistasis

Description. A study of knockout epistasis in Boolean gene regulatory networks, also covered in Section 5.2 [Tower, 2026b].[5] The clone carries 1,267 tracked files, 3 registered plans, 153 ledger events, 13 runs, 131 manuscript claims bound to a run, 7 claim records and 18 quotations.

All Findings. 18: 3 claim, 2 source, 10 record, 3 cosmetic.

Example Finding. Binding the manuscript's numbers to runs required recomputing them, and the recomputation revealed that a property stated for all 28 networks holds only for 27.

Table 24: Knockout epistasis: what each command found in a clone at the pinned commit, and what it could not establish. Exit 0 is a pass, 1 a finding, and 2 nothing to read.

command	exit	what it found	what it could not establish
`prereg`	0	3 plans, each unchanged against the digest recorded when it was frozen	whether a plan was written before the outcomes it governs were seen
`results`	1	chain intact and anchored at 153 events; 8 of 12 recorded files match their digest and 4 carry more than one recorded digest	what share of the manuscript's numbers the 131 claims cover, the manuscript not being in the repository
`citations`	1	18 quotations declared against 7 sources, each carrying a SHA-256; none checked, the sources not being in the repository	whether any quotation occurs in its source
`repro`	2	no evidence manifest	any assertion, none having been declared

K Precedents for Re-execution

The `repro` manifest, `repro.yaml`, lists each command to rerun, the input files the command reads, and the numbers the manuscript prints from its output. `repro reproduce` reruns each listed command in a directory that holds only that command's declared inputs, and then compares every number the manuscript prints from the output with the value at its declared address. Table 25 sets each of its design choices beside the closest earlier system and states the difference. Most choices have a direct precedent. Three do not, in the sources read: an outcome for a rerun whose number could not be read, a label on a step that ran after an upstream failure, and a comparison of a printed value with a declared address in an output file that stays with the project.

[5] https://github.com/elliottower/knockout-epistasis-dynamics, commit cb82384.

Table 25: Design choices of `repro reproduce`, the closest precedent for each, and the difference. Rerunning is common and is specified in many ways; a mechanical comparison against the numbers a manuscript prints, with outcomes that keep a rerun that did not happen apart from a rerun that disagreed, is where the command departs from the systems before it.

Design choice	Closest precedent	Difference
Commands to rerun are declared in a manifest	DVC lists stages that wrap shell commands [Iterative, Inc., 2020]; Snakemake states rules from input files to output files [Köster and Rahmann, 2012]; ReproZip traces one execution [Chirigati et al., 2016]; many ClawBio skills write a replay script when they run [Corpas, 2026].	A regeneration is declared beside the claims that read its output.
Each command runs with only its declared inputs	ReproZip unpacks into a directory, a chroot, a virtual machine or a container [Chirigati et al., 2016]; Code Ocean, Whole Tale and Binder package an environment [Code Ocean, 2026, Brinckman et al., 2019, Jupyter et al., 2018].	A directory of declared inputs and no container: an input that was not declared is absent.
Success is that the printed numbers still hold	A codechecker judges, and identical results are not required [Nüst and Eglen, 2021]; ACM badges ask for agreement within a tolerance [Association for Computing Machinery, 2020]; CORE-Bench accepts answers inside an interval from three reruns [Siegel et al., 2024]; SEER-Repro accepts a relative error of at most 5%, or a difference of at most half a unit in the last decimal the paper reports [Liu et al., 2026]; DVC compares a dependency's contents with their previous version to decide whether a stage is rerun [Iterative, Inc., 2020]; ClawBio gives checksums of selected output files for a reader to check [Corpas, 2026].	Each number the manuscript prints is compared mechanically, and the bytes need not match.
Outcomes are kept apart	A study of notebooks counts separately those whose dependencies failed to install, those that failed in execution, those that finished with different results and those that finished with the same [Pimentel et al., 2019]; ReproScore leaves its outcome score undefined without execution evidence [Samuel et al., 2026]; SEER-Repro labels a perturbed analysis inconclusive, unavailable or not tested, and counts a reproduced value that is missing or unparseable as not reproduced [Liu et al., 2026]; CODECHECK has no process for a failed check [Nüst and Eglen, 2021].	Five outcomes are recorded for every regeneration, and one that was not rerun never reads as reproduced.
A rerun whose number cannot be read is `unchecked`	Paperclip fails a paper whose extracted value is invalid [Generative Expert Labs, 2026]; ScientistOne sorts its extraction misreads by hand [Meng et al., 2026].	The case has its own recorded outcome.
A step after a failure still runs, and is labeled	DVC can skip or run the dependents of a failed stage [Iterative, Inc., 2020]; Snakemake deletes the outputs of an incomplete rule [Köster and Rahmann, 2012].	The dependent step runs over the pinned copy, and its record says so.
Every invocation is appended to a log	Workflow Run RO-Crate packages a run [Leo et al., 2024]; AiiDA records each process in a provenance graph [Huber et al., 2020]; in-toto writes a signed link for each step [Torres-Arias et al., 2019]; DVC keeps the current state in a lock file [Iterative, Inc., 2020].	One append-only file beside the project holds what each invocation observed.
Checking never executes	SLSA verification compares a digest and expectations, and rebuilding is optional [Open Source Security Foundation, 2023]; an in-toto verifier checks signed records of steps [Torres-Arias et al., 2019]; DVC has a dry run [Iterative, Inc., 2020].	`repro verify` and `repro reproduce` are separate commands with separate guarantees.
A printed value is compared with a declared address in an output file	ScientistOne tags each numeric sentence with a log line and compares within 5% [Meng et al., 2026]; when the document is built, Sweave inserts the output of its code chunks and showyourwork regenerates each figure from its script [Leisch, 2002, Luger et al., 2021]; a CODECHECK manifest ties an output file to a figure by a comment [Nüst and Eglen, 2021].	The address is declared, resolved and compared, an ambiguous address is an outcome, and the record stays with the project.

Chain-of-Evidence. ScientistOne [Meng et al., 2026] is an autonomous research system whose writer tags each sentence that holds a number with a line of its experiment log, and whose audit

reruns submitted code and compares the result with the reported score. `repro reproduce` differs from it in five ways. (1) *Who can use it.* The tags exist only for papers that ScientistOne itself writes, and its audit needs a benchmark's own evaluator; the `repro` manifest is written for any project, by a person or an agent, and lists the author's own commands. (2) *What the reader receives.* ScientistOne removes the tags from the final manuscript; the manifest, the pinned inputs and the log of reruns stay in the repository, so a reader repeats the check from a clone. (3) *How a check is decided.* Its audit has a language model read the reported score out of the paper, and two of its four checks are majority votes of a language model; every claim in the manifest is machine-checkable, in that the number the manuscript prints is compared, as a decimal at the precision printed, with the value at its declared address in the rerun output, and no language model takes part. (4) *What is fixed beforehand.* A regeneration runs only on inputs whose bytes match their pins, and a changed input is reported as `not re-run`; the ScientistOne paper describes no digest of an input or an output. (5) *What a result can be.* Its numeric check passes or fails, and its authors sort the misread values by hand; `unchecked`, `failed` and `not re-run` are outcomes of their own, so a check that could not be made is never counted with those that passed. ScientistTwo applies the same audit to its own output and reports every score verified on 49 target problems [Nam et al., 2026]. Its coding agent audits the repository against the manuscript, the rerun in the one audit report the paper reproduces covers the proposed method and not the baseline, and the paper states no tolerance for a score to count as verified.

Reproduction by an agent. SEER-Repro has a coding agent re-derive 2,190 atomic claims of 20 published cancer-registry papers from the registry data, and reproduces 77.4% of them [Liu et al., 2026]. A language model extracts each reproduced value from the agent's report, and deterministic code then applies the tolerance. The two designs start from opposite ends. In SEER-Repro a model extracts the claims from a published paper, the reference is the published number, and the analysis is rewritten by the agent; in the `repro` manifest the author declares each claim and its address, the reference is the project's own output, and the command that is rerun is the author's.

Checklists and badges. The machine learning reproducibility checklist asks an author to state what a paper includes [Pineau et al., 2020], and an ACM badge records that the main results were obtained in a later study by a team other than the authors [Association for Computing Machinery, 2020]. Neither specifies what is rerun or compares a printed number with an output. The `repro` manifest states what is rerun, as a list of commands and the input files each reads, and the log of `repro reproduce` records for every printed number whether the rerun output still holds it.

L Audit: Literature census of 43,462 abstracts

Description. A preregistered literature census of proteome-wide Mendelian randomization papers, in progress.[6] It counts how often a protein–outcome claim is repeated, and how often the repeats rest on overlapping data. The frozen plan is registered on OSF through `prereg register`, and the plan's log records the registration and each amendment. The repository carries 1 frozen plan with 194 log lines, 233 ledger events, 40 runs, 3,508 claim records and 23,939 quotations.

Example Finding. `results verify -files` reports one sealed test file as recorded with 2 different hashes: it was corrected in place and sealed again, against the study's rule that a fix is a new file. No command clears the report.

Table 26: Literature census: what each command found in the repository, and what it could not establish. Exit 0 is a pass, 1 a finding, and 2 nothing to read.

command	exit	what it found	what it could not establish
`prereg`	0	1 plan, unchanged against the digest recorded when it was frozen	whether the plan answers the question
`results`	1	chain intact at 233 events; 373 of 374 recorded files match their digest and 1 carries 2 recorded digests	which run a reported number came from, no number being in a manuscript yet
`citations`	1	13,989 of 13,989 pinned source sentences and 2,167 of 2,167 pinned sentences behind the dataset codes found in their stored sources; 7,783 quotations in the repository's own claims folder unchecked, their sources not being in the repository	whether a found sentence was read correctly
`repro`	0	5 of 5 assertions verified	any count of the census, the assertions covering the search counts the plan states

Application: Use during the study

The census is conducted with Reproducible Science. A census of this size, run by a language model and by coding agents in separate sessions, could not be audited or called confirmatory without the records of Table 27. Each extraction can be checked against its source sentence, and each rule is dated against what had been seen.

Table 27: The record kept at each stage of the census, and the command that keeps it. A review of this kind ordinarily reports one count per stage.

Stage	**Command**	**Record**
plan	`prereg`	one plan frozen before screening, and 194 dated log lines under it, each stating what had been seen when it was written
screening and extraction	`results`	90 seals of scripts and inputs and 40 recorded runs, over 43,462 abstracts screened and 14,053 claim records extracted
verification	`citations`	the source sentence of each of the 14,053 claim records, checked against the stored abstract
dataset coding	`citations`	2,167 pinned sentences behind the codes for 872 dataset names
access to output	`results`	26 access records, each with what was shown and its level

Model extraction. A language model screens 43,462 abstracts and extracts 14,053 claim records from the 2,363 papers it admits. Each record carries the sentence it was drawn from, so `citations verify` decides for every record whether the model quoted the abstract. Of the 14,053 sentences, 13,989 resolve in the stored abstract. Of the 64 that do not, 27 occur twice in the abstract and 37 leave out one word of it. A found sentence establishes nothing about whether the model read it correctly.

[6] Registration: https://osf.io/zae8c. The audit is of the study's repository at commit `62f181c`.

Agent coding. Coding agents assign a dataset to each of 872 dataset names in 434 papers. An agent's code counts only when `citations pin` finds the sentence it rests on in the source and that sentence holds an identifier unique to the coded release. Across four passes of separate sessions, a script applies that rule to every code, and the coordinating agent asks each session for counts and file paths and never for values.

Access to output. Each access to output is a ledger record with a level. Of 26 accesses, 4 are at the level *metadata only*, 19 at *structure seen* and 3 at *outcomes seen*: a registered negative-control result, the text of three evidence sentences printed by accident during a count, and the opening lines of one batch of the second check, printed to learn its format. None shows a quantity either hypothesis concerns. The plan was frozen before all three, so each confirmatory number is still recorded as confirmatory, with the freeze it rests on named beside it.